GreenWave · Prototype Master Plan

Team Cubitx · VISTERA 2026 · what is built, how to run it, how the ambulance alert works, and what comes next

A. What exists right now (rebuilt to the team's mockup and tested end to end on 9 Oct)

- A working web prototype in the folder GreenWave_Prototype: index.html (the app), server.py (a small local server that also downloads YouTube videos), run.bat, README.txt, GreenWave_Demo_Script.pdf (what to say and click, about four minutes). It runs on your laptop with no install beyond Python and Chrome.
- The page follows the team's landing design: headline, a live junction in the hero, six feature cards, and a dark "How the green wave happens" strip. Each card scrolls to a working section: See the traffic, Smarter signals, Connected roads, Emergency first, A smoother commute, Learn every day. The card itself shows that section's live status.
- Junction A runs on a CCTV feed: the bundled sample clip, any YouTube junction video pasted as a link (the local server downloads it and it plays), previously downloaded videos, a file from the PC, or a webcam. Vehicles are detected live in the browser with TensorFlow.js and the COCO-SSD model, and the count on the chosen approach drives the signal.
- Junction B is a second browser tab 400 m down the corridor, running the same adaptive controller. On start it plays its own clip automatically (the next downloaded video, or the sample), so both junctions show live detection.
- Control room is a third tab: the corridor city, both junctions' queues and phases, and a live event feed. Platoons and ambulances move through the city as they travel between junctions.
- Corridor city. Connected roads and the control room draw the North corridor as an isometric 3D city of three junctions: J-01 Market Road (Junction A), J-02 Central Junction (Junction B) and J-03 Hospital Road (simulated timing until a controller is attached). Signal heads, amber queued vehicles, moving platoons and the flashing ambulance are drawn from the live states. Click a junction card to read its last decision; switch 3D view / Top view, drag to pan, zoom with + and –.
- Default video. Every run starts the same fixed CCTV view of a busy Indian four-way crossroads with a signal (YouTube _l99w_CKwCU, kept in the videos folder after the first download). Vehicles are detected in all four approach regions and stopped vehicles at the red are counted as queued.
- Every decision is explained in plain words on screen ("Switched to EW. Reason: the other axis has the longer queue, 9 vs 2"), and logged.
- A fixed-timer mode is one click away, and a 30 s + 30 s comparison button runs both controllers on the same traffic and reports the percentage saved.
- Evidence: Download log (CSV) exports every decision with the queues and average wait at that moment; Download lessons (CSV) exports the learning table.

A2. The three upgrades added on 9 October

- All four approaches from the video. The camera frame is split into regions (top North, bottom South, left West, right East; two-region and whole-frame layouts are available). Every detected vehicle is assigned to a region, so all four queues come from the picture and nothing on Junction A is random while a feed runs.

- Queue, not density. Vehicles are tracked between frames. One that moves less than half a car length per second is counted as queued (yellow box); moving vehicles are green. The signal reads queued counts. For a free-flowing highway clip the count mode can be switched to all vehicles.
- Learning from history. At the end of every NS+EW cycle the junction records the share of demand each axis carried and the average wait, in a table keyed by 10-minute time slot. After two cycles in a slot, the minimum green is biased toward the busier axis (between 5 s and 11 s instead of a flat 8 s). The table is kept in the browser between runs, shown in the Learn every day section, and every switch explanation says when a learned minimum was used.

B. How to run it in the review

1. Make sure the laptop is online for the first run (the detection model downloads once, about 6 MB).
2. Double-click run.bat. It starts a local server and opens three tabs: Junction A, Junction B, Control room.
3. Both junctions start their own clips automatically. In Junction A, Source Sample CCTV clip is already running; The frame is split into four approach regions; green boxes are moving vehicles, yellow boxes are stopped (queued) ones, and all four queues follow the picture. Change the region layout to match your clip.
4. To show your own junction video: set Source to YouTube link, paste the link, click Start feed. The feed card shows download progress, then the video plays with detection. It is kept under Source: Downloaded videos for next time. Junction B can do the same.
5. Arrange Junction B and Control room beside it. Watch B receive platoon messages from A.
6. For a camera-triggered alert, set Source to Ambulance test clip in A and Start feed: about ten seconds in the camera flags the ambulance. Or click Simulate ambulance. A holds a protected green, B shows the red alert with a countdown, pre-clears its green 12 s before arrival, and acknowledges. The control room shows the ambulance moving A to B.
7. In A smoother commute press Run 30 s + 30 s comparison: the fixed timer runs, then GreenWave, on the same traffic, and the two average waits are shown side by side.
8. In Learn every day point at the highlighted row: the NS share of demand recorded for this time slot and the minimum greens now in use.
9. Shortcut: Run 60-second demo in Junction A does steps 3, 6 and 7 by itself with a progress label, so the presenter can talk while it runs. The control room shows corridor totals and a live wait-time chart with the ambulance moment marked.

C. How the ambulance alert reaches the next junction

1. Detect. The camera detector flags a vehicle whose body shows the red cross or light bar: between 5 and 22 percent red pixels with some white, at least 60 px tall, for four frames in a row. The thresholds were measured on the bundled ambulance clip, on which the rule fires reliably about ten seconds in, and checked against the ordinary traffic clips. The test button fires the same event. The pilot replaces the rule with a trained ambulance class and siren detection.
2. Protect locally. Junction A immediately moves to the ambulance's axis, after a 2.5 s amber if needed, and holds that green for 14 s. The safety guard still enforces the minimum green and the all-red clearance.
3. Publish. A sends a message: type ambulance, to B, axis EW, approach W, ETA. ETA = link length / corridor speed + clearance = 400 m / 11 m/s + 10 s, about 46 s. In the prototype the message travels over the browser's BroadcastChannel between tabs; in the pilot it is an MQTT message on

the city network. Same payload, same logic.

4. Prepare. Junction B shows the alert and counts down. At 12 s before arrival it begins pre-clearing: it switches to the EW axis so the queue in front of the ambulance drains before it gets there.
5. Acknowledge. B sends an ack back to A and the control room logs it, so an operator can see the handover happened.
6. Release. Once the ETA has passed plus a margin, B returns to normal adaptive control and counts one ambulance cleared.
7. Chain. With more junctions, B would repeat step 3 towards C. Each junction only needs to know its neighbours, so the corridor scales without a central server.

D. Controller logic, in one paragraph

Every 1/60 s the junction updates queues (simulated arrivals, plus the live camera count on the camera approach), then decides. In adaptive mode it holds the current green at least 8 s and at most 45 s, switches when the other axis has the longer queue or the current axis is empty, and switches regardless when any road has waited more than 75 s (fairness). Switching always passes through 2.5 s of amber. Vehicles clear at one per 1.1 s per lane on green. When an EW green ends, the junction sends the number it released to its neighbour with an arrival ETA, and the neighbour injects that platoon into its own queue when it arrives and biases its next green to meet it. That is the green wave.

E. Architecture of the prototype

Part	In the prototype	In the pilot
Camera	Sample clip, YouTube link, video file or webcam	Existing CCTV RTSP stream
Detection	TensorFlow.js COCO-SSD in the browser	YOLOv8 on a Jetson at the junction
Ambulance recognition	White-and-red colour rule, 3 frames	Fine-tuned ambulance class plus siren audio
Controller	Rule-based adaptive policy with learned minimum greens per time slot	PPO policy trained in SUMO, same guard rules
Junction messaging	BroadcastChannel between tabs	MQTT over the city network
Signal output	On-screen signal heads	Standard controller interface
Dashboard	Control room tab	React dashboard, FastAPI, PostgreSQL log

F. What to say if a judge asks

- Is the camera real? The feed is real CCTV-style traffic video and the boxes are live detections by a neural network running in the browser. The signal on the West approach follows the count. A webcam or the junction's own RTSP stream plugs into the same slot.
- Is the AI real? The detector is a real neural network and the tracker decides which vehicles are queued. The signal controller is the rule-based adaptive policy, and it learns: the minimum greens it uses come from what it recorded at this time of day in earlier runs. The reinforcement-learning policy is trained in SUMO in the next phase and drops into the same slot with the same safety guard.
- How does B know about the ambulance? A publishes a message with the axis and an ETA. B pre-clears 12 s before the ETA and acknowledges. Today the message is a browser channel; in a

pilot it is MQTT. Nothing else changes.

- What if the camera is wrong? Counts are smoothed, an ambulance needs three consecutive frames, and the guard enforces minimum green and all-red clearance no matter what the detector says. If the feed drops, the junction falls back to the timer plan.

G. Next 72 hours after the review

1. Replace the colour rule with a trained ambulance class (about 300 labelled frames are enough to start) and add siren audio detection.
2. Build the junction in SUMO and train PPO; compare with the rule-based policy on the same traffic.
3. Swap BroadcastChannel for a real MQTT broker (Mosquitto) and run Junction B on a second laptop.
4. Record a 60-second demo video of the camera, the alert and the control room for the next round.
5. Replace the per-slot minimum-green bias with the trained policy once SUMO results beat it on the same logs.

H. Known limits of this build

- The detection model downloads from the internet on first load; after that it is cached by the browser.
- COCO-SSD has no ambulance class, so the prototype uses a red-band colour rule tuned on the bundled ambulance clip, or the test button. A very red car could in principle trip it; the four-frame streak and the 22 percent ceiling keep that rare.
- Junction B's traffic is simulated; the camera only feeds A.
- The page is served from localhost by run.bat (server.py) so that video files, YouTube downloads and the webcam work in Chrome. Keep that window open while presenting.
- Live tabs in the background are throttled by the browser; keep the three tabs in separate windows or switch between them during the demo.